GPS Software Design Document

(SDD)

Revision: 01

Date: 17 / 06 / 2026

Author: illi Eisenberg

Table of Contents

1. Introduction

1.1 Purpose

This document describes the detailed software design of the GPS Driver. The purpose of this document is to define the internal software structure, interfaces, data models, algorithms, and implementation concepts used by the driver. It provides the information required to understand, maintain, extend, and port the software implementation.

This document complements the GPS Software Architecture Document (SAD) by describing how the architectural requirements are implemented within the software.

1.2 Scope

This document covers the detailed design of the GPS driver implementation, including:

- Software components and their responsibilities
- Internal data structures
- Public and internal APIs
- Protocol abstraction mechanisms
- Protocol detection logic
- NMEA sentence processing
- Parser implementation concepts
- UART transport integration
- Ring buffer handling
- Memory management considerations
- FreeRTOS integration details
- Error handling mechanisms

Hardware-specific implementation details that are not directly related to the GPS driver design are outside the scope of this document.

1.3 References

The following documents and specifications were used during the design and implementation of the GPS driver:

- GPS Software Architecture Document (SAD)
- NMEA 0183 Protocol Specification
- CASIC Multimode Satellite Navigation Receiver Protocol Specification
- MediaTek PMTK Command Packet User Manual
- STM32 Reference Manual
- STM32 HAL Driver Documentation
- FreeRTOS Documentation

Additional project-specific requirements and application-level specifications may also be referenced when applicable.

2. Design Overview

2.1 Design Goals

The GPS driver was designed to provide a flexible, portable, and maintainable software solution for GNSS receiver integration.

The primary design goals are:

- Support multiple GPS receiver protocols through a unified interface
- Separate protocol-specific logic from core driver functionality
- Minimize dependencies between software modules
- Support efficient UART communication using DMA
- Enable easy integration with FreeRTOS-based applications
- Simplify future protocol and parser extensions
- Minimize runtime memory consumption
- Avoid dynamic memory allocation

The design emphasizes modularity and long-term maintainability while remaining suitable for resource-constrained embedded systems.

2.2 Design Principles

The driver implementation follows several key design principles:

2.2.1 Separation of Concerns

Each software module is responsible for a specific functionality such as protocol handling, sentence parsing, transport management, or data storage.

2.2.2 Protocol Abstraction

Protocol-specific implementations are isolated behind a common interface, allowing the application and core driver logic to remain independent of the underlying GPS protocol.

2.2.3 Static Memory Allocation

All major data structures and buffers are statically allocated to ensure deterministic memory usage and eliminate runtime allocation overhead.

2.2.4 Reusability

The design aims to maximize reuse of common functionality such as checksum validation, NMEA processing, and UART transport services.

2.2.5 Extensibility

The internal architecture allows new protocols, commands, and NMEA sentence parsers to be added with minimal impact on existing functionality.

2.3 Layered Design

The GPS driver is organized into several logical layers, each providing a well-defined responsibility.

2.3.1 Application Interface Layer

Provides the public API used by higher-level software components to control and access GPS functionality.

2.3.2 Core Driver Layer

Manages driver initialization, protocol selection, command dispatching, and overall driver state management.

2.3.3 Protocol Layer

Implements protocol-specific commands and receiver control operations for supported GPS protocols.

2.3.4 NMEA Processing Layer

Handles sentence validation, routing, parsing, and extraction of GPS information.

2.3.5 Transport Layer

Provides UART-based communication services, DMA integration, and ring buffer management for data reception and transmission.

2.5.6 Data Layer

Stores parsed GPS information and maintains internal data structures used by the driver and higher software layers.

3. Software Components

3.1 Core Driver Layer

The Core Driver Layer serves as the central control component of the GPS driver.

Its primary responsibilities are:

- Driver initialization
- Protocol selection and binding
- Driver state management
- Command dispatching
- Coordination between software layers

This layer provides the main entry point for the application and exposes the public GPS API. The Core Driver Layer is protocol-independent and does not contain protocol-specific command implementations.

3.2 Protocol Layer

The Protocol Layer contains protocol-specific implementations used to communicate with supported GPS receivers.

Each supported protocol provides:

- Command generation
- Receiver configuration functions
- Receiver control functions
- Protocol-specific message formatting

Current protocol modules include:

- PMTK Protocol Module
- PCAS Protocol Module

The Protocol Layer is accessed through a common protocol interface, allowing the Core Driver Layer to operate independently of the selected receiver protocol.

3.3 NMEA Processing Layer

The NMEA Processing Layer is responsible for processing incoming NMEA sentences received from the GPS receiver.

Its responsibilities include:

- Sentence validation
- Checksum verification
- Sentence identification
- Sentence routing
- Data extraction

The layer separates sentence recognition from sentence parsing, allowing individual sentence parsers to remain independent and modular. Supported sentence types are processed through dedicated parser functions.

3.4 Transport Layer

The Transport Layer provides the communication interface between the GPS driver and the physical UART peripheral.

Its responsibilities include:

- UART transmission
- UART reception
- DMA integration
- Ring buffer management
- Data buffering

This layer isolates hardware communication details from higher software layers and provides a consistent interface for data exchange.

The Transport Layer is designed to be replaceable if a different communication mechanism is required in the future.

3.5 Data Management Layer

The Data Management Layer stores information extracted from GPS receiver messages.

Its responsibilities include:

- Position data storage
- Satellite information storage
- Firmware information storage
- Runtime status information storage

Parsed information is stored in dedicated data structures that can be accessed by higher software layers through the driver API. This layer acts as the central repository for all GPS-related runtime data maintained by the driver.

4. Data Structures

4.1 GPS_Context_t

The `GPS_Context_t` structure serves as the main runtime context of the GPS driver.

It contains the driver configuration, runtime state information, and protocol-specific function bindings required for operation.

4.1.1 Responsibilities

- Store driver initialization status
- Maintain current protocol selection
- Store requested communication parameters
- Hold protocol callback function pointers
- Provide a central runtime context for the driver

4.1.2 Stored Information

- Driver state information
- Active protocol information
- UART communication settings
- Protocol-specific operation handlers

This structure is created during driver initialization and remains active throughout the driver lifetime.

4.2 GPSData_t

The `GPSData_t` structure stores positioning and navigation information extracted from NMEA sentences.

It represents the current navigation solution provided by the GPS receiver.

4.2.1 Responsibilities

- Store position information
- Store navigation status
- Store movement information
- Store date and time information
- Store fix status information

4.2.2 Stored Information

- Latitude
- Longitude
- Direction indicators
- Speed
- Course over ground
- Altitude
- Date
- Time
- Position validity status
- Fix quality information

This structure is continuously updated as new navigation sentences are received and parsed.

4.3 GPSsats_t

The `GPSsats_t` structure stores satellite visibility and signal quality information.

The structure is populated from GSV sentence data and represents the current satellite environment observed by the receiver.

4.3.1 Responsibilities

- Store visible satellite information
- Store satellite signal levels
- Store satellite identifiers
- Store satellite positioning information

4.3.2 Stored Information

- Total visible satellites
- Satellite PRN identifiers
- Signal-to-noise ratio (SNR) values
- Option: Elevation values
- Option: Azimuth values

The structure may contain information from multiple GSV sentences required to complete a full satellite report.

4.4 GPSfw_t

The `GPSfw_t` structure stores firmware and hardware identification information obtained from the receiver.

Its contents depend on the capabilities of the connected GPS module and the protocol being used.

4.4.1 Responsibilities

- Store firmware identification information
- Store hardware identification information
- Store manufacturer information
- Store build information

4.4.2 Stored Information

- Firmware version
- Release information
- Build identifier
- Chip information
- Manufacturer information

This information is typically updated only when version-related commands are executed.

4.5 Protocol Interface Structures

Protocol interface structures define the abstraction layer between the Core Driver Layer and protocol-specific implementations.

These structures contain function pointers that provide a common interface for all supported protocols.

4.5.1 Responsibilities

- Abstract protocol-specific implementations
- Provide uniform command execution
- Allow runtime protocol selection
- Enable protocol extensibility

4.5.2 Supported Operations

Typical operations include:

- Version retrieval
- Receiver restart
- Hot start
- Cold start
- Standby mode control
- Baud rate configuration
- NMEA sentence configuration

By using protocol interface structures, the Core Driver Layer remains independent of any specific GPS protocol implementation.

5. Public APIs

This section defines the public API provided by the GPS driver.
These APIs are used by higher-level application components to control and retrieve information from the GPS subsystem.

All APIs are designed to be:

- Non-blocking where possible
- Safe for FreeRTOS usage
- Independent of GPS protocol implementation
- Consistent across different receiver types

5.1 Driver Initialization APIs

These APIs are responsible for initializing and preparing the GPS driver for operation.

GPS_Init()

Initializes the GPS driver, including:

- UART and transport layer initialization
- Ring buffer setup
- Protocol binding initialization
- Internal context setup

Returns:

- Success or failure status

5.2 Configuration APIs

These APIs configure runtime behavior of the GPS receiver and driver.

GPS_Set_Baud()

Sets the UART baud rate of the GPS receiver. Used after baud detection.

- Sends protocol-specific command to the receiver
- Reconfigures local UART interface accordingly
- Updates internal driver context

5.3 Control APIs

These APIs control the operational state of the GPS receiver.

GPS_StartRMC()

Enables periodic RMC sentence output from the receiver.

GPS_StopRMC()

Disables RMC sentence output.

GPS_Standby()

Places the GPS receiver into low-power standby mode.

5.4 Data Access APIs

These APIs provide access to parsed GPS information stored in the driver.

GPS_GetVersion()

Retrieves firmware and hardware version information from the receiver.

Returns structured firmware data stored in `GPSfw_t`.

General Data Access Behavior

- APIs return the latest available parsed data
- Data is read from internal driver-managed structures
- No direct UART or protocol interaction is required from the application layer

6. Protocol Abstraction Design

6.1 Protocol Interface Concept

The GPS driver supports multiple receiver protocols through a protocol abstraction mechanism. Instead of allowing the application to directly interact with protocol-specific implementations, all protocol operations are accessed through a common runtime interface stored in the driver context.

This approach allows the Core Driver to remain independent of any specific GPS protocol implementation.

Typical protocol operations include:

- Start RMC output
- Stop RMC output
- Set baud rate
- Retrieve firmware version
- Restart receiver
- Enter standby mode
- Hot start
- Cold start

All protocol-specific implementations expose a common set of operations regardless of the underlying protocol.

6.2 Function Pointer Binding

Protocol abstraction is implemented using function pointers stored inside the driver context structure. Example:

```
struct GPS_Context
{
    /* function table */
    bool (*start_rmc)  (GPS_Context_t *ctx);
    bool (*stop_rmc)   (GPS_Context_t *ctx);
    bool (*start_gsv)  (GPS_Context_t *ctx);
    bool (*stop_gsv)   (GPS_Context_t *ctx);

    bool (*get_version)(GPS_Context_t *ctx);
    bool (*hot_start)  (GPS_Context_t *ctx);
    bool (*cold_start) (GPS_Context_t *ctx);
    bool (*set_baud)   (GPS_Context_t *ctx);
    bool (*standby)    (GPS_Context_t *ctx);
    bool (*restart)    (GPS_Context_t *ctx);

};
```

During initialization, the selected protocol assigns its implementation functions to the corresponding function pointers.

Example:

```
ctx->start_rmc = PMTK_StartRMC;
ctx->stop_rmc  = PMTK_StopRMC;
ctx->get_version  = PMTK_Get_Version;
ctx->set_baud     = PMTK_Set_Baud;
```

or

```
ctx->start_rmc = PCAS_StartRMC;
ctx->stop_rmc  = PCAS_StopRMC;
ctx->get_version  = PCAS_Get_Version;
ctx->set_baud     = PCAS_Set_Baud;
```

The application and Core Driver always invoke the generic interface and never call protocol-specific functions directly.

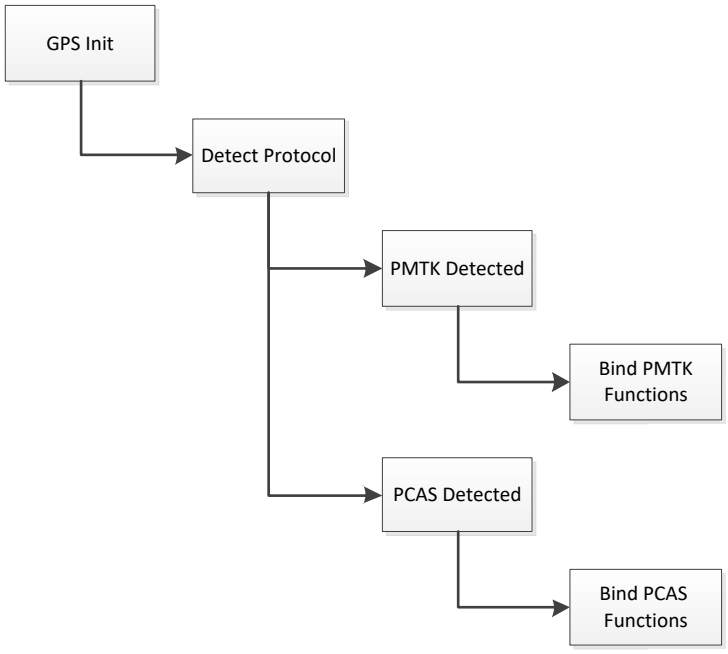
6.3 Protocol Selection Logic

Protocol selection occurs during driver initialization. The initialization sequence performs protocol identification and determines which protocol implementation should be used. After successful identification:

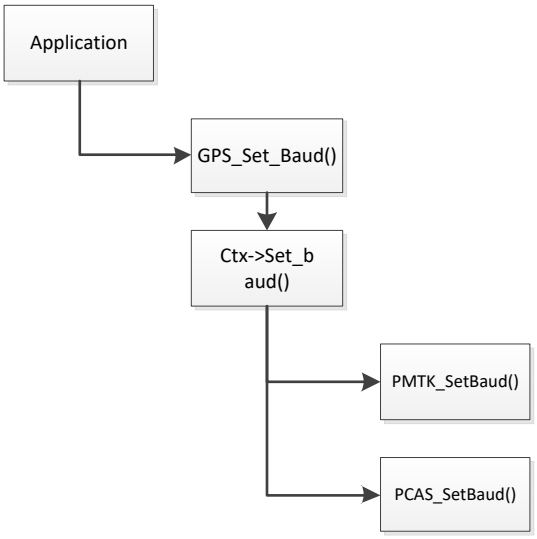
1. The active protocol is stored in the driver context.
2. Protocol function pointers are assigned.
3. The driver becomes ready for normal operation.

The remainder of the system operates through the abstract interface and is unaware of the selected protocol.

Binding Flow



After binding is completed:



The same mechanism is used for all protocol-dependent operations.

After protocol detection and binding, the driver may perform additional protocol-specific configuration steps such as normalizing the receiver baud rate to the application's preferred operating speed.

6.4 Adding a New Protocol

Adding support for a new GPS protocol requires implementing a new protocol module that conforms to the existing protocol interface.

The implementation typically includes:

- Protocol command definitions
- Command generation functions
- Receiver configuration functions
- Version query support
- NMEA output configuration functions

Required steps:

1. Create a new protocol source module.
2. Implement all required protocol handlers.
3. Add protocol identification logic.
4. Bind the new handlers during initialization.
5. Verify compatibility with existing NMEA processing modules.

Because the Core Driver communicates only through the abstract interface, adding a new protocol requires minimal modifications to the existing codebase.

7. Protocol Detection Design

7.1 Detection Objectives

The GPS driver supports multiple receiver protocols and must determine the active protocol before normal operation can begin.

The protocol detection mechanism is responsible for:

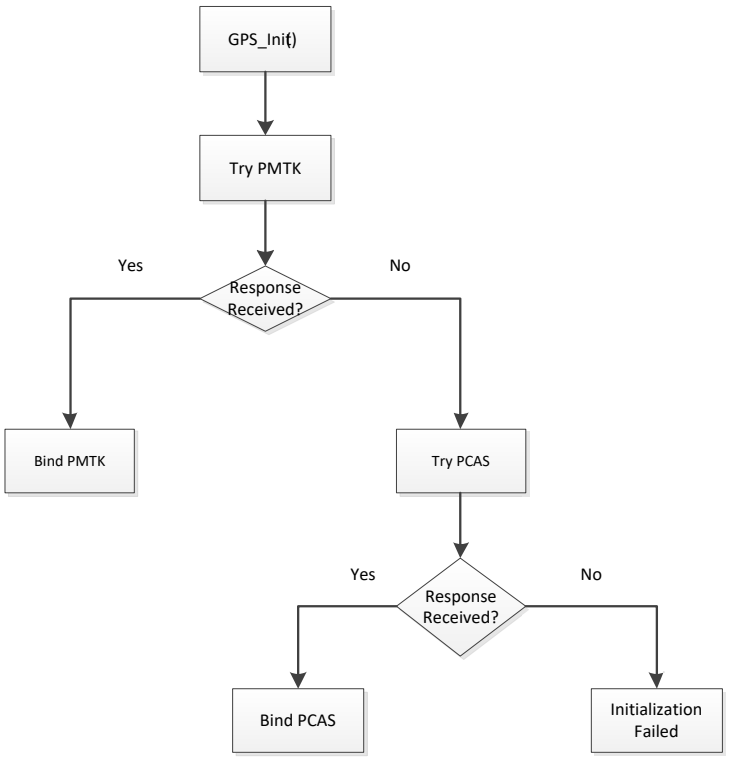
- Identifying the connected GPS receiver protocol
- Selecting the correct protocol implementation
- Binding protocol-specific handlers
- Preventing unsupported protocol operation

Protocol detection is performed only during driver initialization.

7.2 Detection Sequence

The detection process is implemented as a sequential probing algorithm. The driver transmits a protocol-specific command and waits for a valid response from the receiver. If a valid response is received, the corresponding protocol implementation is selected and bound to the driver context. If no response is received, the next supported protocol is tested.

Detection Flow



The detection sequence may be extended in the future to support additional protocols.

7.3 Response Validation

A received message is not considered valid solely because data was received on the UART interface. The driver performs protocol-specific validation before accepting a response. Validation may include:

- Header verification
- Command identifier verification
- Message structure verification
- Checksum verification
- Expected response matching

Only responses that successfully pass validation are accepted as evidence of protocol identification.

Invalid or unrelated messages are ignored.

7.4 Failure Handling

Protocol detection fails when none of the supported protocol probes produce a valid response.

Possible failure causes include:

- Unsupported GPS receiver
- Incorrect baud rate
- UART communication failure
- Receiver startup delay
- Invalid or corrupted responses

When protocol detection fails:

- No protocol handlers are bound
- Driver initialization is aborted
- The initialization function returns a failure status

The application is responsible for handling initialization failures and determining whether a retry mechanism should be performed.

8. NMEA Processing Design

8.1 Sentence Reception

Incoming NMEA data is received through the UART interface using DMA. The DMA engine continuously transfers received bytes into the reception buffer without CPU intervention. Received data is copied into the ring buffer, which serves as a temporary storage area between the UART transport layer and the NMEA processing layer.

The ring buffer allows asynchronous reception and prevents data loss when multiple NMEA sentences arrive consecutively.

Sentence boundaries are identified using the standard NMEA terminator sequence:

<CR><LF>

Each completed sentence is extracted from the ring buffer and forwarded for validation.

8.2 Sentence Validation

Before a sentence is processed, its structure is verified. The validation stage ensures that the received message follows the expected NMEA format.

Typical validation checks include:

- Sentence begins with '$'
- Sentence length is within valid limits
- Sentence contains a checksum separator ('*')
- Sentence terminates correctly
- Required fields are present

Sentences that fail structural validation are discarded immediately.

Only valid NMEA sentences continue to the checksum verification stage.

8.3 Checksum Verification

After structural validation, the NMEA checksum is verified.

The driver calculates the XOR checksum over all characters located between:

$ and *

The calculated checksum is compared with the checksum value transmitted within the sentence.

Example:

```
$GPRMC,...*6A
```

If the calculated checksum matches the transmitted checksum, the sentence is considered valid.

Checksum failures indicate corrupted or incomplete data and result in immediate rejection of the sentence.

8.4 Sentence Routing

Validated sentences are routed according to their sentence identifier. The router extracts the sentence type from the NMEA header and selects the appropriate parser.

Examples:

```
GPRMC → RMC Parser
GPGSV → GSV Parser
GPGSA → GSA Parser
GPTXT → TXT Parser
```

The routing layer isolates sentence identification from parser implementation and simplifies future expansion. Unsupported sentence types are ignored.

8.5 Parser Dispatching

After routing, the selected parser function is executed.

Each parser is responsible for:

- Extracting relevant fields
- Converting textual values into internal data types
- Updating the corresponding GPS data structures
- Handling missing or invalid fields

Examples:

- RMC
- parser updates position, speed, course, date, and time
- GSV parser updates satellite information and signal strength
- GSA parser updates fix information
- TXT parser updates firmware and receiver information

Parsers operate independently and update only the data structures associated with their sentence type.

This design allows new sentence parsers to be added without modifying the reception or routing mechanisms.

9. Parser Design

9.1 RMC Parser

The RMC parser processes the NMEA Recommended Minimum Navigation Information sentence.

The parser extracts:

- UTC Time
- Status (Valid / Invalid)
- Latitude
- Longitude
- Direction indicators (N/S, E/W)
- Speed Over Ground
- Course Over Ground
- UTC Date

The parser converts latitude and longitude from DMM (Degrees and Decimal Minutes) format into internal Decimal Degree representation.

Speed values are converted from knots into kilometers per hour.

When the sentence status indicates invalid navigation data, position, speed, and course values are cleared.

The parsed information is stored in the GPSData_t structure.

9.2 GSV Parser

The GSV parser processes satellite visibility information.

The parser extracts:

- Total number of GSV messages
- Current message number
- Total satellites in view

For each satellite entry:

- Satellite ID (PRN)
- Elevation angle
- Azimuth angle
- Signal-to-Noise Ratio (SNR)

Since a complete satellite report may span multiple GSV sentences, the parser accumulates information across consecutive messages until the entire report has been received.

Parsed satellite information is stored in the GPSsats_t structure.

9.3 GSA Parser

The GSA parser processes GNSS DOP and Active Satellite information.

The parser extracts:

- Fix mode

The following fields are not parsed due to the need for "Fix" mode only:

- Fix type
- Active satellite identifiers
- Position Dilution of Precision (PDOP)
- Horizontal Dilution of Precision (HDOP)
- Vertical Dilution of Precision (VDOP)

The most important output of the parser is the current navigation fix status.

The fix value is stored within the GPSData_t structure and is used by higher software layers to determine navigation validity.

Typical values include:

```
0 = No Fix
1 = 1D Fix
2 = 2D Fix
3 = 3D Fix
```

9.4 TXT Parser

The TXT parser processes receiver information messages. TXT messages are primarily used to retrieve receiver identification data and firmware information. The parser identifies supported TXT message categories and extracts their associated values.

Typical categories include:

```
SW = Software Release
TB = Build Date
IC = Chip Identification
MA = Manufacturer
```

Unknown TXT categories are ignored.

Parsed information is stored in the GPSfw_t structure.

9.5 Firmware Information Parsing

Firmware information may be distributed across multiple TXT messages.

Each TXT message contributes a specific piece of information to the firmware database.

Examples:

```
$GPTXT,...,SW=URANUS5,V5.3.0.0
$GPTXT,...,TB=2020-03-26,13:25:12
$GPTXT,...,IC=AT6558R-5N-32-1C580901
$GPTXT,...,MA=CASIC
```

The parser updates only the field associated with the received TXT category. Previously parsed firmware fields remain unchanged. The resulting firmware information structure may contain:

- Software Release
- Firmware Version
- Build Date
- Chip Identifier
- Manufacturer

This approach allows firmware information to be assembled incrementally as TXT messages are received.

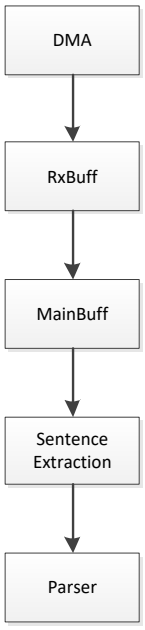
10. Ring Buffer Design

10.1 Buffer Architecture

The driver uses a two-stage buffering mechanism to decouple UART reception from NMEA processing. Incoming UART data is first stored in a DMA reception buffer (RxBuf). After each DMA reception event, newly received bytes are transferred into the main ring buffer (MainBuf).

The ring buffer provides temporary storage for incoming NMEA data and allows the reception layer and parsing layer to operate independently.

Data Flow:



The ring buffer architecture ensures that consecutive NMEA sentences can be received without blocking parser execution.

10.2 DMA Integration

UART reception operates in DMA mode. The DMA controller continuously transfers incoming bytes into the reception buffer without requiring CPU intervention for each received character. When a DMA reception event occurs, the driver determines which portion of the reception buffer contains new data and copies only the newly received bytes into the ring buffer.

This design minimizes CPU overhead and allows efficient handling of continuous NMEA traffic.

The DMA layer is responsible only for data reception and does not perform sentence parsing or protocol processing.

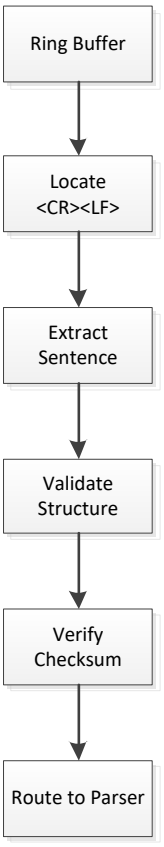
10.3 Sentence Extraction

NMEA sentences are extracted from the ring buffer after a complete sentence has been received. Sentence boundaries are identified using the standard NMEA termination sequence:

<CR><LF>

Whenever a complete sentence is detected, the internal sentence counter is incremented. The processing layer removes complete sentences from the ring buffer one at a time and forwards them for validation and parsing.

Typical processing flow:



This approach allows multiple NMEA sentences to accumulate in the buffer and be processed sequentially.

10.4 Overflow Considerations

The ring buffer has a fixed size and therefore cannot grow dynamically.

If incoming data arrives faster than it can be processed, the buffer may eventually become full.

To reduce the probability of overflow:

- DMA reception operates continuously.
- Sentence processing is triggered as soon as complete sentences are available.
- NMEA traffic can be reduced by disabling unnecessary sentence types.
- Only required NMEA messages are enabled during normal operation.

The driver assumes that the application configures the GPS receiver output rate appropriately for the selected buffer size and processing capacity.

Under normal operating conditions, the ring buffer is expected to process incoming data faster than new data is generated.

11. Memory Management

11.1 Static Memory Usage

The GPS driver uses a fully static memory model. All buffers, data structures, and internal objects are allocated at compile time. No memory is allocated or released during runtime.

The static allocation approach provides:

- Deterministic memory usage
- Predictable execution behavior
- Simplified debugging
- Elimination of memory fragmentation risks

The memory footprint of the driver remains constant throughout system operation.

11.2 Context Storage

Driver state information is maintained within a dedicated context structure.

The context stores:

- Initialization status
- Selected protocol
- Active baud rate
- Protocol function bindings
- Driver configuration parameters

The context structure is statically allocated and exists for the entire lifetime of the application. This design guarantees that protocol bindings and runtime configuration remain continuously available without requiring dynamic allocation.

11.3 Parser Buffers

NMEA sentence processing uses fixed-size local and global buffers.

Typical parser storage includes:

- DMA reception buffer
- Ring buffer storage
- Temporary sentence extraction buffer
- Parser working buffers

Parser buffers are sized according to the maximum expected NMEA sentence length. Buffer sizes are defined at compile time and remain fixed during execution. This approach prevents unexpected memory growth and simplifies resource estimation.

11.4 Runtime Memory Considerations

The driver does not use dynamic memory allocation mechanisms.

The following operations are intentionally avoided:

```
malloc()
calloc()
realloc()
free()
```

All runtime processing is performed using preallocated memory regions.

Advantages of this approach include:

- No heap dependency
- No memory fragmentation
- Predictable RAM consumption
- Improved system stability
- Suitability for long-running embedded systems

The driver is designed to operate continuously without requiring memory cleanup or garbage collection mechanisms.

As a result, memory behavior remains deterministic regardless of system uptime.

12. FreeRTOS Integration Design

12.1 Task Notifications

The GPS driver utilizes FreeRTOS task notifications to signal the availability of newly received NMEA data. When a complete NMEA sentence is detected, the reception layer notifies the processing task that data is ready for extraction and parsing.

Task notifications are used instead of polling in order to:

- Reduce CPU utilization
- Minimize task wakeups
- Improve system responsiveness
- Eliminate unnecessary periodic checks

Notification events are represented as bit masks, allowing multiple event sources to be combined into a single notification mechanism.

Example event sources include:

- GPS sentence available
- New command received
- Processing request

The receiving task evaluates the notification flags and executes the appropriate processing path.

12.2 Queue Interaction

The GPS driver itself does not directly communicate with application-level queues. Parsed GPS information is written into internal driver data structures. Higher software layers may retrieve this information and forward it through FreeRTOS queues when required.

Typical queue usage includes:

- Application command delivery
- GPS data distribution
- GUI updates
- Bluetooth communication
- GPX recording services

The separation between parsing and queue transmission keeps the driver independent from application-specific logic.

12.3 UART Synchronization

UART access is protected through a synchronization mechanism to prevent concurrent access from multiple tasks.

Only one software component may perform a UART transaction at a time.

The synchronization layer ensures:

- Serialized command transmission
- Protection against overlapping UART writes
- Consistent protocol communication
- Safe coexistence with other UART users

The synchronization object is acquired before transmitting a command and released when the transaction is complete.

This mechanism prevents corruption of command sequences and guarantees predictable UART behavior.

12.4 ISR Interaction

The UART reception path includes interaction between interrupt service routines and FreeRTOS tasks. DMA reception events occur within interrupt context.

The ISR performs only minimal processing:

- Detect newly received data
- Update buffer state
- Detect completed sentences
- Generate task notifications

Computationally intensive operations such as:

- Sentence extraction
- Validation
- Checksum verification
- NMEA parsing

are intentionally deferred to task context.

This design minimizes interrupt latency and keeps ISR execution time short and deterministic.

The overall processing sequence is:

```
UART Interrupt / DMA Event
        ↓
Update Reception Buffers
        ↓
Detect Complete Sentence
        ↓
Send Task Notification
        ↓
GPS Processing Task
        ↓
Sentence Parsing
```

This separation ensures efficient utilization of CPU resources while maintaining reliable NMEA reception.

13. Error Handling Design

13.1 Invalid NMEA Sentences

Before a received sentence is processed, its basic structure is validated.

The validation stage verifies that the sentence:

- Begins with the '$' character
- Contains a checksum separator ('*')
- Falls within the supported length limits
- Contains the minimum required fields

Sentences that fail structural validation are immediately discarded. No parser is executed for invalid sentences.

This prevents malformed or incomplete data from propagating into the internal GPS data structures.

13.2 Checksum Errors

After structural validation, the NMEA checksum is verified. The checksum is calculated by performing an XOR operation on all characters located between:

$ and *

The calculated value is compared against the checksum transmitted within the sentence. If the values do not match:

- The sentence is rejected
- No parser is executed
- Existing GPS information remains unchanged

Checksum verification protects the driver against corrupted UART traffic and partially received sentences.

13.3 Protocol Detection Failures

During initialization, the driver attempts to identify the protocol supported by the connected GPS receiver.

Each supported protocol is queried according to the protocol detection sequence.

If no valid response is received from any supported protocol:

```
PMTK Detection Failed
        ↓
PCAS Detection Failed
        ↓
Initialization Failed
```

The driver remains uninitialized, and protocol binding is not performed.

The initialization function reports failure to the caller, allowing the application to decide whether to retry initialization or enter an error state.

No protocol-specific commands may be issued until successful protocol detection has completed.

13.4 UART Communication Failures

The driver assumes that all communication with the GPS receiver occurs through a UART transport layer.

Communication failures may include:

- No response received
- Incomplete responses
- Unexpected data
- UART hardware errors
- DMA reception failures

The driver validates all received data before processing.

If communication errors occur:

- Invalid data is discarded
- Existing GPS information remains unchanged
- The driver continues monitoring the UART interface for future valid messages

Transient communication failures do not require driver restart and do not affect previously parsed GPS information.

This approach allows the receiver to recover automatically once valid communication resumes.

14. Performance Considerations

14.1 DMA Utilization

UART reception is implemented using DMA in order to minimize CPU involvement during data acquisition.

Incoming GPS data is transferred directly from the UART peripheral into the DMA reception buffer without requiring software intervention for each received byte.

Benefits of this approach include:

- Reduced interrupt frequency
- Lower CPU overhead
- Improved reception reliability
- Continuous reception of NMEA streams

The CPU is only involved when new data becomes available and requires processing. This allows the driver to efficiently handle continuous GPS traffic while preserving processor resources for other system functions.

14.2 CPU Load Reduction

The driver is designed to minimize processor utilization during normal operation. Several mechanisms contribute to this goal:

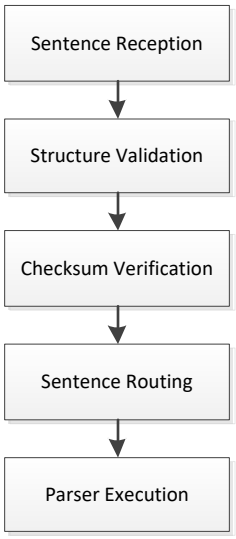
- DMA-based UART reception
- Ring buffer decoupling
- Event-driven processing
- Task notification wake-up mechanism

The processing task remains blocked while no GPS data is available. Parser execution occurs only after complete NMEA sentences have been received. This approach eliminates continuous polling and reduces unnecessary task execution. As a result, CPU usage remains proportional to actual GPS traffic.

14.3 Parsing Efficiency

NMEA sentence processing follows a staged validation sequence.

Each sentence passes through:



Invalid sentences are discarded as early as possible. This prevents unnecessary parser execution and avoids processing overhead for corrupted or unsupported data. Parser implementations operate directly on sentence fields and update only the relevant data structures.

The design avoids expensive processing operations and minimizes runtime overhead.

14.4 Memory Footprint

The driver uses a fixed memory model with compile-time allocation.

Memory consumption consists primarily of:

- DMA reception buffer
- Ring buffer storage
- GPS context structure
- Parsed data structures
- Temporary parser buffers

No dynamic memory allocation is used.

The driver does not perform:

```
malloc()
calloc()
realloc()
free()
```

Because all memory is allocated statically, RAM usage remains constant throughout system execution.

This behavior provides predictable resource consumption and makes the driver suitable for resource-constrained embedded systems.

15. Portability Considerations

15.1 MCU Dependencies

The GPS driver was developed and tested on STM32 microcontrollers.

Most GPS processing components are platform-independent, including:

- NMEA sentence validation
- Checksum verification
- Sentence routing
- NMEA parsers
- Protocol abstraction layer
- GPS data structures

Hardware-specific dependencies are limited to:

- UART configuration
- DMA configuration
- Timing services
- Interrupt configuration

By isolating hardware access within dedicated transport modules, the majority of the driver can be reused on different MCU platforms with minimal modification.

15.2 FreeRTOS Dependencies

The driver is designed to operate in a FreeRTOS environment.

Current FreeRTOS integrations include:

- Task notifications
- Queue communication
- Synchronization primitives
- ISR-to-task signaling

The core GPS processing logic does not directly depend on FreeRTOS APIs. Only the integration layer requires adaptation when migrating to a different operating system or a bare-metal environment. This separation reduces the effort required to port the driver to alternative execution environments.

15.3 UART Dependencies

The transport layer abstracts the physical communication channel used by the GPS receiver.

The current implementation utilizes:

`STM32 UART + STM32 DMA`

The remaining driver components interact only through transport layer interfaces.

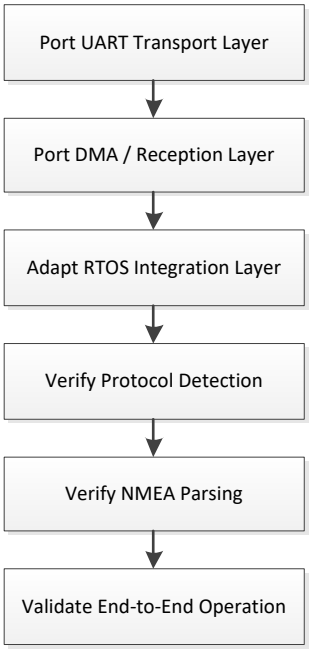
Porting the driver to another platform typically requires replacing:

- UART initialization
- UART transmission functions
- UART reception functions
- DMA handling routines

No modifications are required in the parser or protocol layers.

15.4 Porting Strategy

Porting the driver to a new platform should follow a layered approach. Recommended migration sequence:



The parser layer, protocol abstraction layer, and GPS data structures are intentionally platform-independent and typically require no changes.

This design minimizes porting effort and allows the driver to be reused across different microcontrollers, operating systems, and embedded platforms.

16. Future Design Extensions

16.1 Additional Protocols

The protocol abstraction layer was designed to support multiple GPS vendor protocols. New protocols can be integrated by implementing protocol-specific command handlers and binding them to the protocol interface structure.

Typical future protocol candidates may include:

- u-blox proprietary protocol support
- Quectel proprietary protocol support
- MediaTek protocol extensions
- Vendor-specific command sets

No modifications to the NMEA processing layer are expected when adding new protocol implementations.

16.2 Additional NMEA Parsers

The current implementation supports a subset of NMEA sentence types required by the application. Additional parsers may be added to support new functionality.

Examples include:

- GGA (Global Positioning System Fix Data)
- VTG (Course Over Ground and Ground Speed)
- GLL (Geographic Position)
- ZDA (Date and Time)
- GST (Position Error Statistics)

New parsers can be integrated through the existing sentence routing mechanism without affecting existing parsers.

16.3 Event-Based Updates

The current design stores parsed information within dedicated data structures that are later consumed by higher software layers. Future versions may introduce event-based data distribution mechanisms.

Examples include:

- Parser-generated notifications
- Publish/subscribe data delivery
- Event groups
- Callback-based updates

Such mechanisms may reduce polling requirements and improve responsiveness in systems containing multiple GPS data consumers.

16.4 Advanced Logging Support

Future versions of the driver may include enhanced diagnostic and logging capabilities.

Potential features include:

- NMEA traffic logging
- Parser activity logging
- Protocol detection logs
- Communication error statistics
- Performance monitoring counters

Advanced logging can simplify debugging, field diagnostics, and long-term system monitoring while maintaining separation from the core GPS processing logic.